

ROS 2 Benchmark Report - 13-07-26 - 08:07

This document presents a comprehensive benchmark report evaluating the performance characteristics of a ROS 2 application across various ROS Middleware (RMW) configurations to analyze different metrics like latency, CPU and RAM usage.

The report details a series of test cases, including:

- Topic-based communication under varying message sizes, single and multi-process setups, and long-duration runs.
- Service and Action communication, testing the influence of client numbers and long-duration runs.
- Idle system resource usage and RAM usage scalability with an increasing number of processes.

Each test uses a different configuration file that defines the RMWs to be tested, their respective communication settings (e.g., ipc_on, ipc_off, loaned) and the topologies involved.

The following RMWs were tested: **cyclonedds**, **fastrtps**, **zenoh**.

System Information:

- **ROS Version:** jazzy
- **System Executor:** unknown
- **Architecture:** aarch64

1. Topic Tests

1.a. Payload Size Impact on Single-Process Performance

This test evaluates how CPU and latency are affected by payload size in single-process applications, using various RMW configurations. It uses messages with fixed and dynamic array sizes of 10b, 100kb, 1MB, and 4MB, while keeping the publishing frequency constant at 200 Hz.

The test is configured using the *single_process_pub_sub.conf* file. The topology involves a single publisher node (pub_node_sp) and a single subscriber node (sub_node_sp) communicating over a single topic.

Ideally, the tendency approximation curve in the following test cases should remain constant. Mathematically, this implies an r-squared value of 0 and a slope of 0 for the linear regression. However, to account for natural variance, we will consider values within an acceptable threshold as valid.

Latency Analysis

The following table shows the results of a linear regression analysis for the different RMW (ROS Middleware) configurations. The columns are:

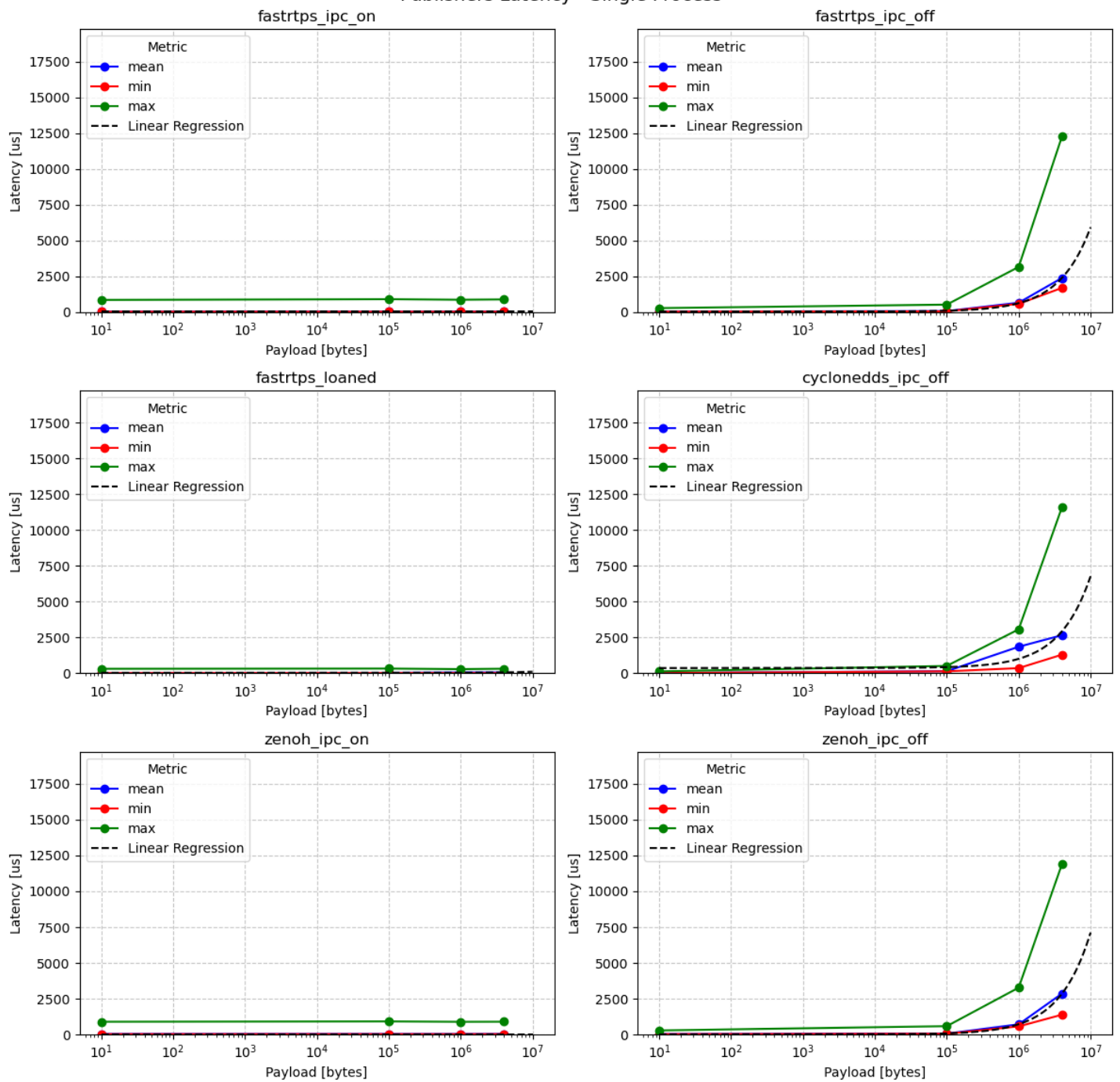
- **r-squared:** The coefficient of determination, indicating how well the linear model fits the data. A value close to **1** suggests a strong correlation, while a value near **0** indicates a weak or no correlation.
- **Slope:** The rate of change in the dependent variable with respect to the independent variable. A slope close to **0** suggests minimal variation in the observed trend.

RMW	r-squared	slope
fastrtps_ipc_on	0.000202	-9.003961822896158e-07
fastrtps_ipc_off	0.931786	0.013423003404248091
fastrtps_loaned	0.209289	0.002161504530801583
cyclonedds_ipc_off	0.932244	0.015048565252805882

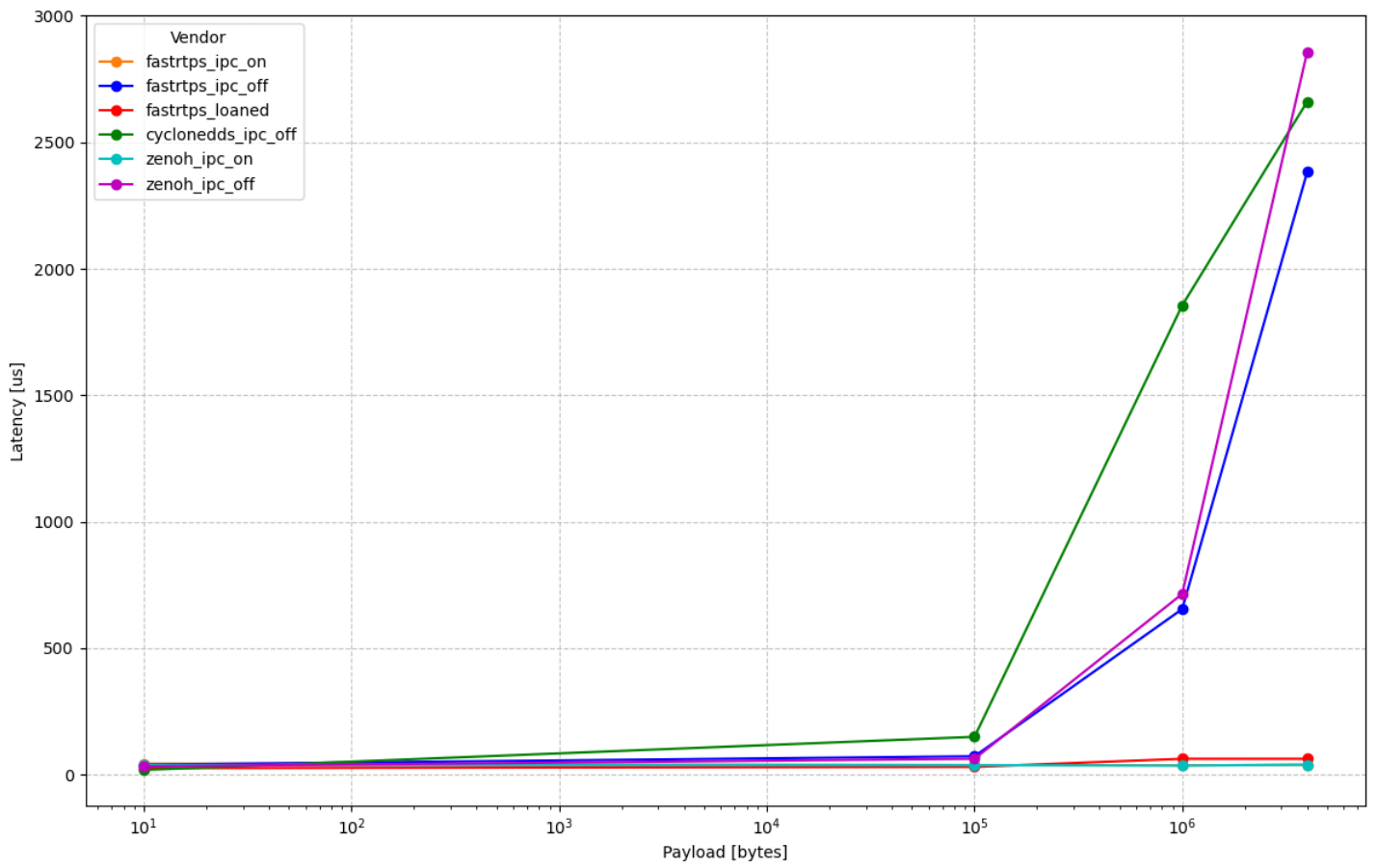
zenoh_ipc_on	0.001840	3.2481485782302127e-06
zenoh_ipc_off	0.921228	0.015308998697967919

Note:: The linear regression trend in the following figures may appear curved in the graph due to the logarithmic scale applied to the x-axis. This transformation compresses larger values, causing a visually nonlinear effect, even though the regression itself remains a straight-line approximation in linear space.

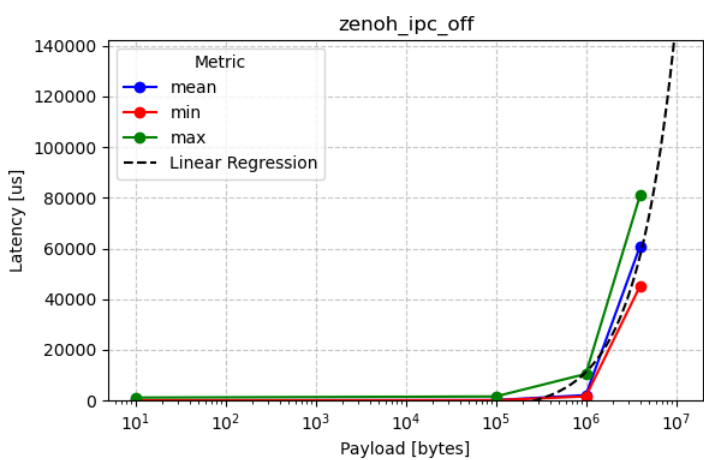
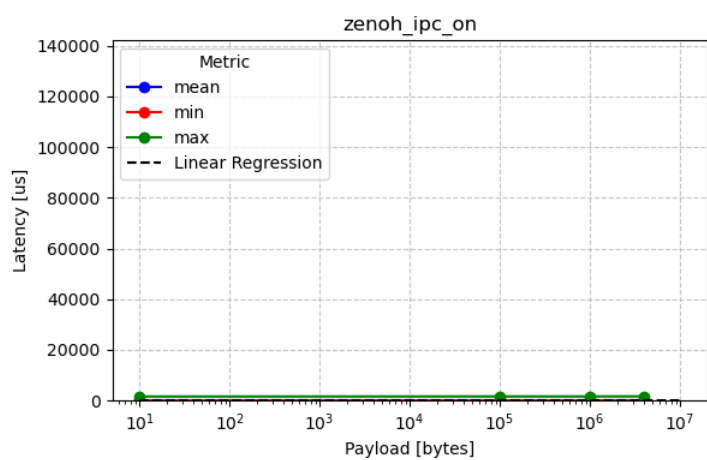
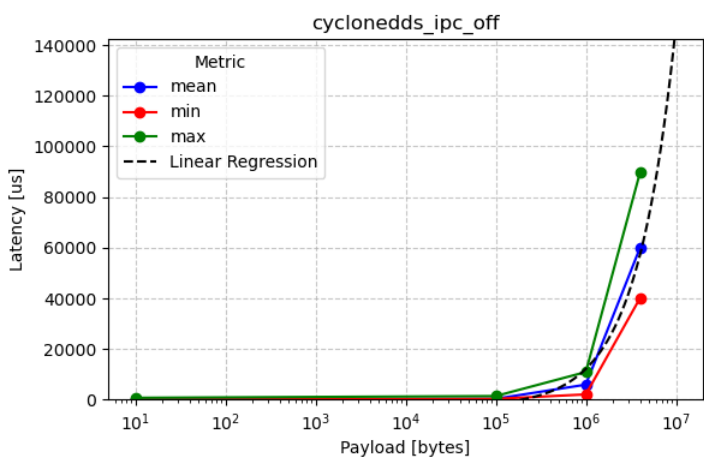
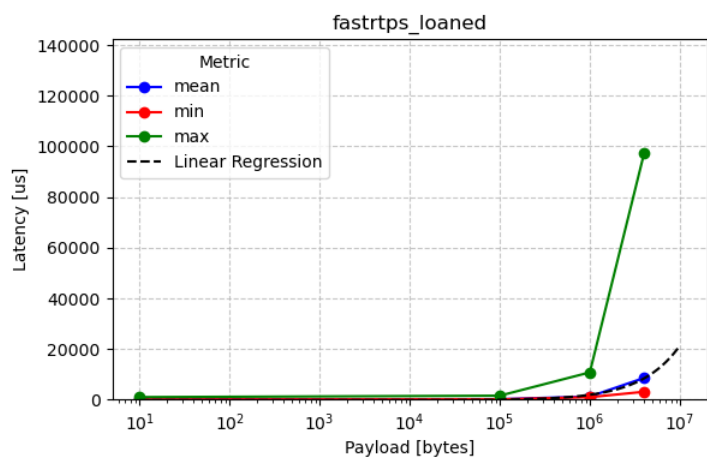
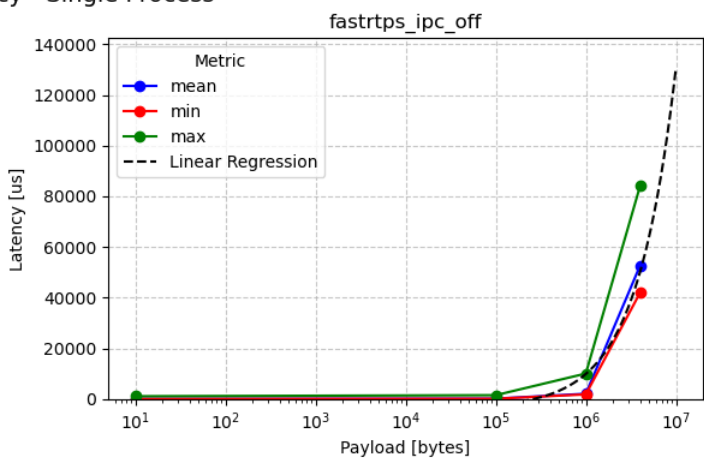
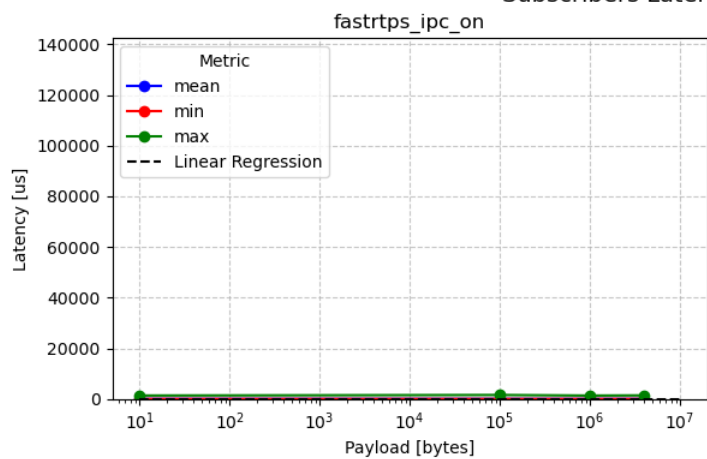
Publishers Latency - Single Process



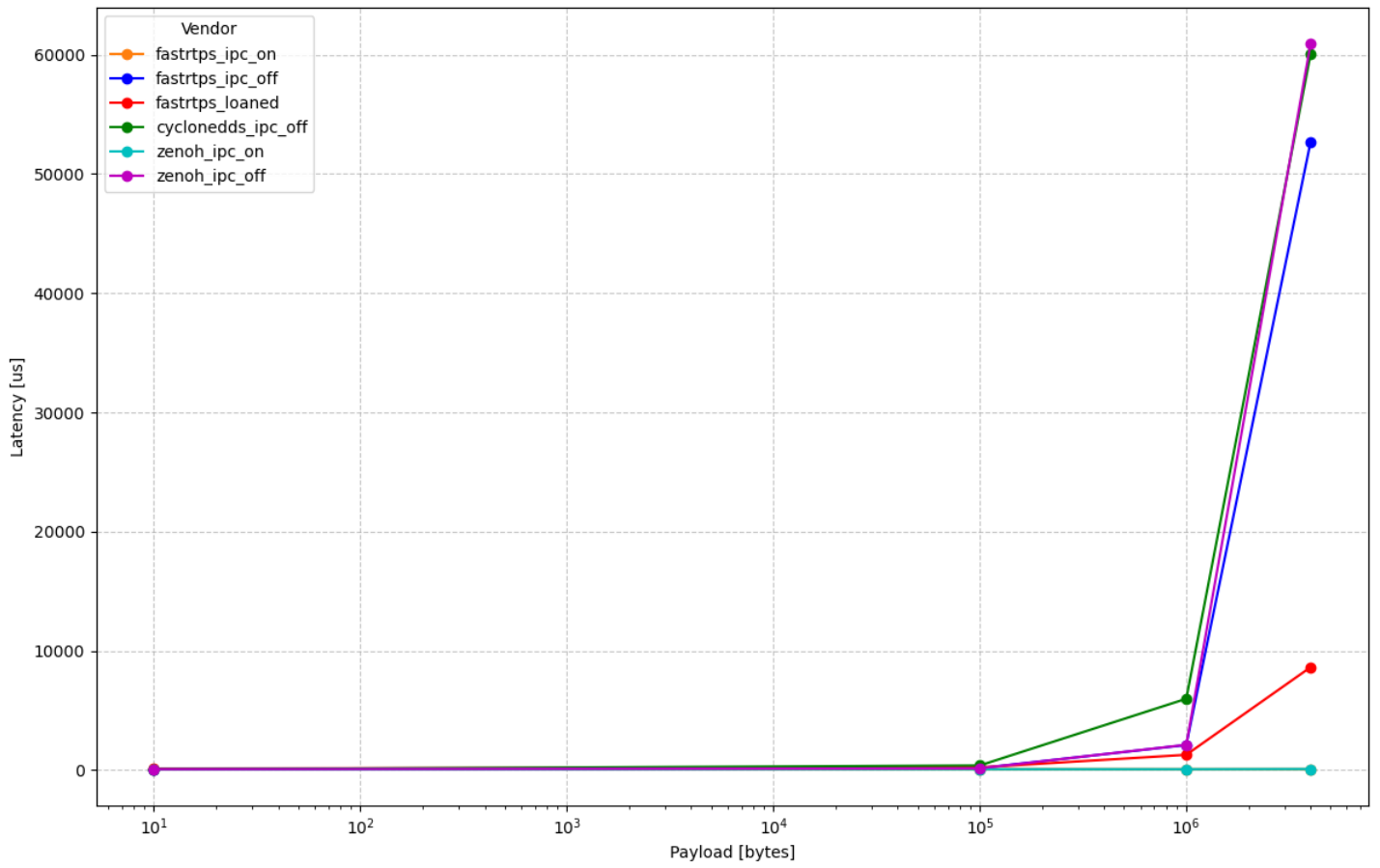
Publishers Latency - Single Process - Mean comparison



Subscribers Latency - Single Process



Subscribers Latency - Single Process - Mean comparison



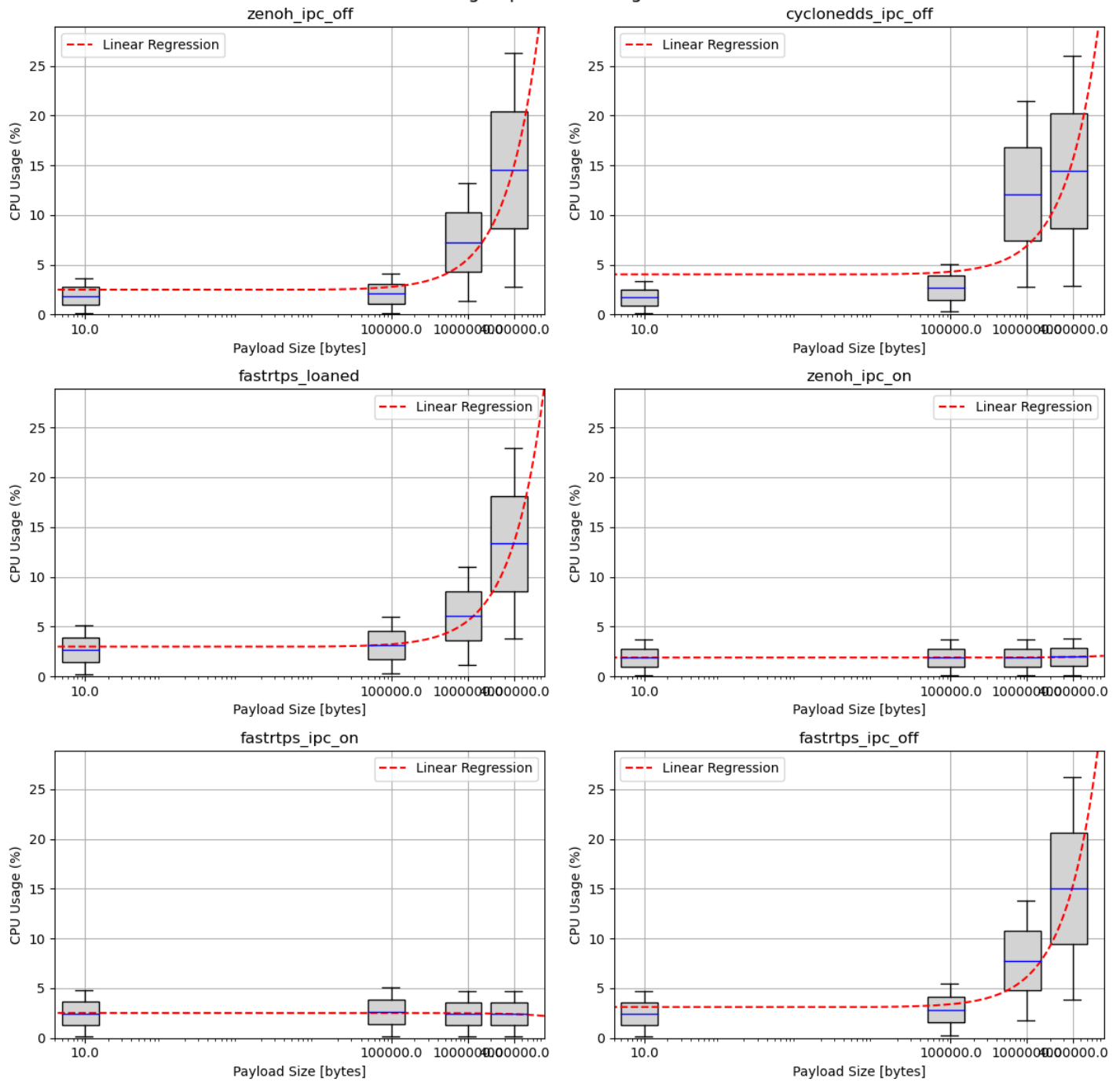
CPU Usage Analysis

The following table shows the results of a linear regression analysis for the different RMW (ROS Middleware) configurations. The columns are:

- **r-squared:** The coefficient of determination, indicating how well the linear model fits the data. A value close to **1** suggests a strong correlation, while a value near **0** indicates a weak or no correlation.
- **Slope:** The rate of change in the dependent variable with respect to the independent variable. A slope close to **0** suggests minimal variation in the observed trend.

RMW	r-squared	slope
zenoh_ipc_off	0.358061	3.121766719499378e-06
cyclonedds_ipc_off	0.251784	2.9202871986826e-06
fastrtps_loaned	0.357245	2.6284831686488144e-06
zenoh_ipc_on	0.000265	1.804756814086452e-08
fastrtps_ipc_on	0.000431	-3.003531779641689e-08
fastrtps_ipc_off	0.361674	3.089616194530096e-06

CPU Usage - pub-sub - Single Process



For information on the graphic visit the [Boxplot documentation](#)

1.b. Payload Size Impact on Multi-Process Performance

This test evaluates how CPU and latency of a ROS 2 application are affected by the size of messages for multi-process applications on the same host, specifically in the case of POD (Plain Old Data) types. The test utilizes messages with fixed payload sizes of 10b, 100kb, 1MB, and 4MB.

The test is configured using the *pub-sub_multi_process.conf* file. This configuration defines paired topologies for each of the payloads, pub_200hz_PAYLOAD and sub_PAYLOAD, which are varied to change the payload size (10b, 100kb, 1MB, 4MB).

Each topology pub_200hz_PAYLOAD includes a publisher node (pub_node) publishing to a single topic with a stamped10b message type at 200 Hz, while the sub_PAYLOAD topology includes a subscriber node (sub_node) listening to the same topic.

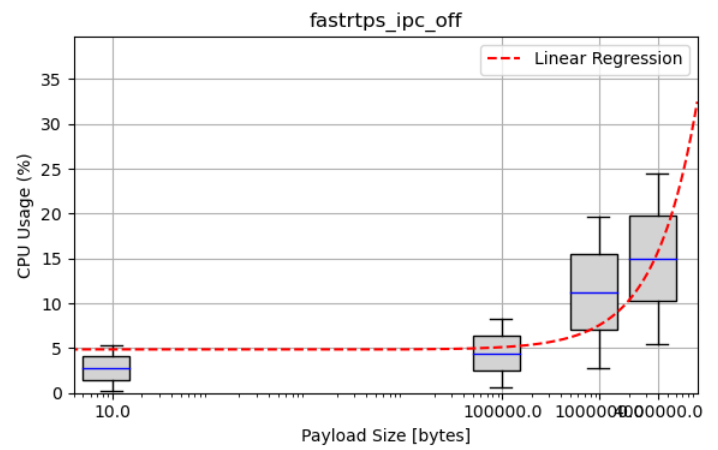
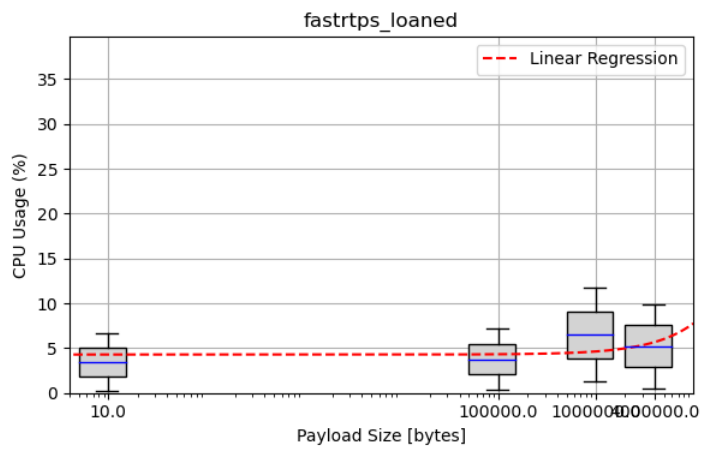
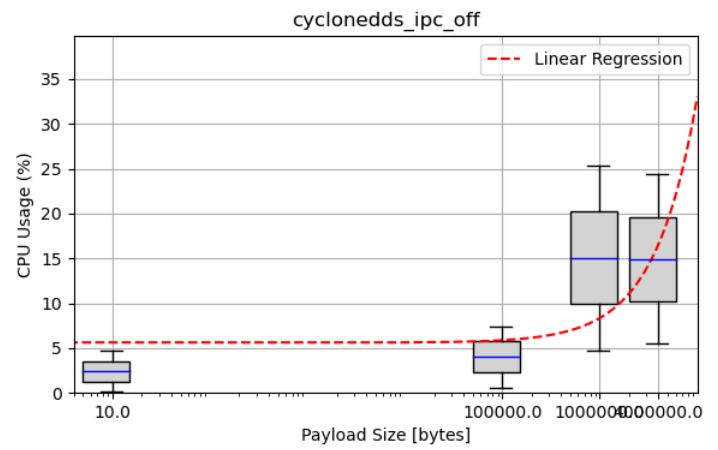
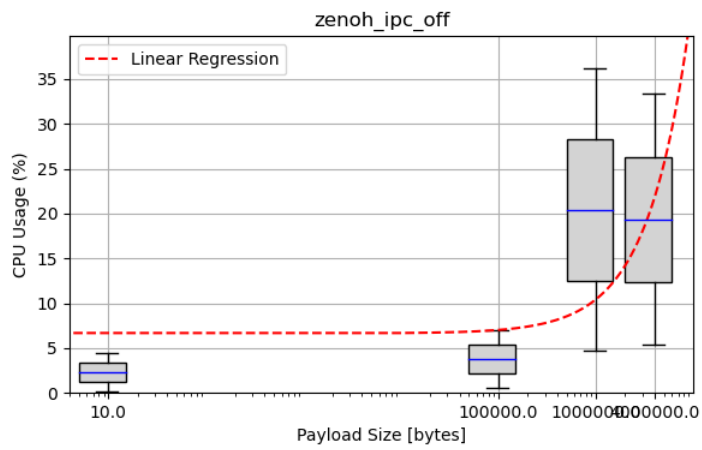
Latency

RMW	r-squared	slope
fastrtps_ipc_off	0.495591	0.01871977012585099
fastrtps_loaned	0.371520	0.004536232409586116
cyclonedds_ipc_off	0.935402	0.054207023111313486
zenoh_ipc_off	0.970087	0.014216955074480163

CPU Usage

RMW	r-squared	slope
zenoh_ipc_off	0.202028	3.779760952150207e-06
cyclonedds_ipc_off	0.225084	2.7428693744392896e-06
fastrtps_loaned	0.016237	3.466745134795711e-07
fastrtps_ipc_off	0.284630	2.7638437079236034e-06

CPU Usage - pub-sub - Multi Process



1.c. Comparison of Multi-Process vs. Single-Process Performance

This test compares the CPU and latency of a multi-process single-host against an equivalent single-process application when using POD message types.

This case follows a statistical approach using the p-value to determine the strength of evidence against the null hypothesis. In the context of this test, the null hypothesis states that a multi-process application using RMW has equal or higher CPU usage and latency compared to an equivalent single-process application.

If the p-value is low (typically ≤ 0.05), there is enough evidence to reject the null hypothesis and conclude that multi-process latency is significantly lower.

On the other hand, if the p-value is high (> 0.05), you fail to reject it, meaning there isn't enough statistical evidence to claim that multi-process performs better than single-process.

Latency

PubDur/fastrtps_ipc_off

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
10b	24.222	75.36	22.148	38.716	1.0
100kb	134.42	338.104	36.434	72.478	1.0
1mb	1467.649	2507.945	247.016	653.791	1.0
4mb	8843.758	13783.099	1956.106	2383.096	0.9999999999999999

PubDur/fastrtps_loaned

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
10b	106.181	54.745	21.805	24.562	0.9999424304318787
100kb	42.314	52.77	22.326	30.393	0.9999999999084112
1mb	333.483	107.545	17.203	62.114	0.9718106589847305
4mb	203.406	125.405	28.708	61.657	0.9999904942777603

PubDur/cyclonedds_ipc_off

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
10b	81.06	132.05	10.548	17.746	1.0
100kb	157.551	688.605	29.681	149.02	1.0
1mb	1707.962	6715.897	820.225	1854.522	1.0
4mb	5533.683	25807.744	2140.339	2660.651	1.0

PubDur/zenoh_ipc_off

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
10b	17.651	53.87	21.051	31.637	1.0
100kb	74.768	414.33	40.328	62.313	1.0
1mb	710.044	3442.57	255.562	713.766	1.0
4mb	1965.347	18151.273	1692.999	2857.763	1.0

SubLat/fastrtps_ipc_off

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
10b	87.805	215.924	77.229	81.095	1.0
100kb	316.525	739.51	103.408	154.418	1.0
1mb	11205.647	7986.103	773.153	2094.199	0.999999999996166
4mb	65594.648	77347.935	6170.041	52669.137	0.997645447461347

SubLat/fastrtps_loaned

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
10b	358.701	326.533	69.241	121.985	0.999999999999162
100kb	602.678	468.82	111.206	216.154	0.9999999890576762
1mb	1167.308	1636.528	949.882	1274.179	0.9995904609446792
4mb	19261.411	18097.555	13649.74	8621.365	0.9999999155325131

SubLat/cyclonedds_ipc_off

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
10b	186.896	282.505	53.59	48.358	1.0
100kb	284.311	1090.025	80.636	379.134	1.0
1mb	13425.92	52004.743	2689.793	5973.643	1.0
4mb	49628.862	215575.25	9555.214	60032.631	0.999999999876221

SubLat/zenoh_ipc_off

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
10b	104.16	355.42	78.498	63.388	1.0
100kb	365.261	1038.98	111.145	136.602	1.0
1mb	822.548	18938.523	811.264	2112.109	1.0
4mb	4321.745	55045.577	6517.956	60915.214	9.945216769653742e-11

CPU Usage

Note: For multi-process CPU usage, the reported metrics represent the combined CPU consumption of both the publisher and subscriber processes. This ensures a fair comparison against the single-process case, where all operations run within a single instance.

zenoh_ipc_off

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
4mb	24.735	33.433	11.727	14.551	0.8353483846214247
10b	4.242	4.629	1.757	1.862	0.8029489746487888
100kb	6.414	7.431	1.962	2.104	0.869277551150456
1mb	28.144	36.024	5.939	7.281	0.9212052572119257

cyclonedds_ipc_off

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
4mb	21.814	33.155	11.588	14.44	0.8580343464639602
10b	4.707	5.024	1.622	1.711	0.8266725538378236
100kb	6.956	8.151	2.388	2.683	0.8535142151474223
1mb	23.137	33.078	9.348	12.107	0.8826843438042933

fastrtps_loaned

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
4mb	14.567	19.273	9.566	13.359	0.6843652410922709
10b	6.573	7.05	2.481	2.671	0.8109866230849714
100kb	7.029	7.693	2.855	3.161	0.8008749861516342
1mb	9.099	10.812	4.948	6.051	0.7421810318039285

fastrtps_ipc_off

payload	multi_std_dev	multi_mean	single_std_dev	single_mean	p_value
4mb	21.869	32.519	11.174	15.061	0.842628834999966
10b	5.29	5.753	2.282	2.421	0.7933003579788448
100kb	7.665	8.925	2.624	2.851	0.8554912396707601
1mb	17.058	22.238	6.049	7.784	0.8706283530146579

1.d. Combined Communication Transports.

This test evaluates the performance of a ROS 2 application when combining multiple communication transports for a topic. To evaluate this hypothesis, we compare two different application configurations using a z-test to analyze Latency and CPU usage.

The first setup involves a single publisher sending messages on a single topic to three subscribers: one in the same process, one in a different process on the same host, and one on a remote host. This configuration is defined by the *mix_process_pub_sub.conf* file and a topology where a *pub_node_sp* publishes to a topic and a *sub_node_sp* subscribes to the same topic.

The second setup uses the *mix_process_pub_sub_multiple_topics.conf* file, where separate publishers and topics are used for each of the three subscribers (same process, different process on same host, remote).

The z-test used for this analysis is a statistical test used to determine if there is a significant difference between the means of two groups.

A low p-value (typically <0.05) indicates strong evidence that the difference suggests potential performance degradation. On the other hand, a high p-value supports the hypothesis by indicating that any observed differences in performance are not statistically significant.

The following tables show the results of the z-test for both latency and CPU usage.

Latency

CPU Usage

1.e. Long-Running Performance Stability

This test evaluates the stability of CPU, latency, and RAM usage over an extended period (greater than 10 minutes).

The test is run using the *single_process_pub_sub_long.conf* configuration file and uses the Sierra Nevada topology, which represents a light 10-node system with multiple topics.

The metrics are sampled every 10 seconds. The first and last 30 seconds of the test are excluded to remove initialization and shutdown effects. Each point represents the mean value within a 10-second bin, helping to smooth out short-term fluctuations.

A stable system should show relatively flat trends, with variations staying within the 10% coefficient of variation threshold. Any significant deviations or increasing trends may indicate performance degradation, resource leaks, or instability in the RMW implementation.

Latency

CPU Usage

1.f. CPU Overhead with Remote Subscription

This test aims to evaluate the CPU overhead between publishing 1MBps of data across 50 topics (with a variety of different message types and publish frequencies), and when a remote host subscribes to all these topics.

The remote connection behavior involves a different script to be executed on the remote host which launches a docker that subscribes to all these topics.

The configuration file used for this test is *single_process_pub_sub_multiple_topics.conf*, and uses the topology_50_topics_2000Hz which defines a single publisher node with 50 publishers, each sending stamped1mb messages on different topics at various frequencies. And a corresponding subscriber node is configured to subscribe to all 50 topics.

CPU Usage

2. Services Tests

2.a CPU Usage and Service Client Count.

This test evaluates the CPU usage under varying numbers of connected service clients. A single service server is deployed alongside a varying number of clients (1, 2, 5, 10, and 20). In each test scenario, only a subset of the clients actively sends service requests, while the others remain idle by using a really high periods. The goal is to verify that the CPU usage is only affected by the number of active clients, and not by the total number of clients connected.

The single-process test uses all components within the same process, while the multi-process test runs each client and server as separate processes. The tests are configured using the *single_process_cli_srv_multiple_clients.conf* and *multi_process_cli_srv_multiple_clients.conf* files respectively.

An example topology used for this test is *5_cli_srv_100kb*, where a *server_node* provides a *stamped100kb* service, and a *multi_node* includes 4 clients configured to send requests at a very high period (600000 ms), effectively making them idle.

The tendency approximation curve should remain constant. This implies an r-squared value of 0 and a slope of 0 for the linear regression. However, to account for natural variance, we will consider values within an acceptable threshold as valid.

Single-Process

Multi-Process

2.b Long-Running Service Performance Stability.

This test assesses whether the CPU, latency, and RAM of a ROS 2 application remain constant over the duration of a long-running test (greater than 10 minutes).

The metrics are sampled every 10 seconds. And the first and last 30 seconds of the test are discarded to avoid initialization and shutdown artifacts.

The test is configured using the `single_process_cli_srv_long.conf` file, which uses a graph topology `cli_srv_architecture` that includes multiple service servers and multiple service clients.

Latency

CPU Usage

RAM Usage

3. Actions Tests

3.a CPU Usage and Action Client Count.

This test evaluates the CPU usage under varying numbers of connected action clients. A single service server is deployed alongside a varying number of clients (1, 2, 5, 10, and 20). In each test scenario, only a subset of the action clients actively sends service requests, while the others remain idle by using a really high periods. The goal is to verify that the CPU usage is only affected by the number of active clients, and not by the total number of clients connected.

The test is configured using the *single_process_actions_multiple_clients.conf* file. An example topology used for this test is `5_cli_srv_100kb`, where a `server_node` provides a `stamped100kb` service, and a `multi_node` includes 4 clients configured to send requests at a very high period (600000 ms), effectively making them idle.

The tendency approximation curve should remain constant. This implies an r-squared value of 0 and a slope of 0 for the linear regression. However, to account for natural variance, we will consider values within an acceptable threshold as valid.

3.b Long-Running Action Performance Stability.

This test assesses whether the CPU, latency, and RAM of a ROS 2 application remain constant over the duration of a long-running test (greater than 10 minutes).

The metrics are sampled every 10 seconds. And the first and last 30 seconds of the test are discarded to avoid initialization and shutdown artifacts.

The test is configured using the `single_process_actions_long.conf` file, which uses a graph topology `cli_srv_architecture` that includes multiple action servers and multiple action clients.

Latency

CPU Usage

RAM Usage

4.a. CPU Usage in Idle state

This test evaluates the idle CPU usage when no messages are being actively published or received, and no user-space periodic tasks are running. To simulate this, publishers and subscribers are created with an extremely long publishing period, set to 1 hour, ensuring that no actual messages are transmitted during the test duration.

The test is configured using the *single_process_pub_sub_idle.conf* file, which sets the *pub_sub_idle* topology. This topology defines an *idle_pub_node* with a publisher sending stamped100kb messages at an extremely low frequency (3600000 Hz, simulating 1 hour period), and an *idle_sub_node* subscribing to the same *idle_topic*.

The test runs for at least 1 minute to avoid measurement bias caused by CPU spikes typically observed during node startup or shutdown.

4.b RAM usage of a ROS 2 application should scale linearly with the number of processes.

The test compares RAM usage across configurations with 1, 2, 5, and 10 processes, where each process runs a single node. It expects the RAM usage to scale linearly with the number of processes.

The test is configured using the *multi_process_scalability_pub_sub.conf* file. It utilizes *topology_multi_1_topics_1000hz* for 1, 2, 5, and 10 topics, respectively, to simulate varying numbers of processes.

Linear regression is used to evaluate whether the observed RAM usage aligns with this expected linear scaling behavior.